/mcp

MCP servers: /mcp [status] | /mcp discover | /mcp connect | /mcp disconnect | /mcp help

Overview

`/mcp` reports the state of Model Context Protocol servers, launches discovery, and switches servers on or off. The bare form (the `status` token lands in the same arm) prints a server table from the current config and runtime state; `/mcp discover` scans for available servers and opens an interactive selection dialog; `/mcp connect <id>` and `/mcp disconnect <id>` enable or disable a named server live.

Whether a server is actually started is durable, user-owned state - it is not just a side effect of being listed in `ragent.json`. The choice is persisted in a **global ledger** (`<global state dir>/mcp_state.json`) so a server disabled once stays disabled in every project, and so a plugin-contributed server (which has no entry in `ragent.json` at all) can still be turned off. A server id **absent** from the ledger is enabled, so a newly added server - written into `ragent.json` or bridged from a plugin's `mcpServers` section - starts enabled with no extra step. A `mcp.<id>.disabled: true` entry in `ragent.json` always disables the server, whatever the ledger says.

Only `discover`, `connect <id>`, `disconnect <id>`, and `help` are real subcommands. There is no `/mcp call` subcommand - tool calls to MCP servers go through the `mcp_tool` tool in chat. The individual tool names are the model-facing surface and are not enumerated by `/mcp`; the full per-server inventory with the registry name each tool is callable under (`<tool> -> mcp_<server>_<tool>`) is available via `/plugins list --mcp`.

Syntax

```
/mcp                      # server status table (status token behaves the same)
/mcp status               # same table
/mcp discover             # scan for MCP servers, open selection dialog
/mcp connect <id>         # enable and connect a server now, persisting the choice
/mcp disconnect <id>      # disable and disconnect a server now, persisting the choice
/mcp help                 # usage help
```

Options / Subcommands

Form	Description
`/mcp`	Print the MCP server table
`/mcp status`	Same as the bare form
`/mcp discover`	Run `McpClient::discover()` and open the interactive dialog
`/mcp connect <id>`	Enable `<id>`, connect it live, and register its tools now
`/mcp disconnect <id>`	Disable `<id>`, disconnect it live; the choice persists globally
`/mcp help`	Print usage help

Examples

Show the server table:

```
/mcp
```

Scan for servers and pick one interactively:

```
/mcp discover
```

Enable and connect a server without restarting:

```
/mcp connect filesystem
```

Disable and disconnect a server (the choice survives a restart and applies to every project):

```
/mcp disconnect filesystem
```

Print the usage help:

```
/mcp help
```

Call an MCP tool from chat (the supported path):

```
mcp_tool(server: "filesystem", tool: "read_file", arguments: {"path": "README.md"})
```

Output

Bare form (`From: /mcp`, `MCP Servers:`):

- With no servers configured, an advisory line suggests running `/mcp discover` to scan for available servers and then adding them to the `mcp` section of `ragent.json` to activate them; enabled plugins that declare an `mcpServers` section are bridged automatically.
- Otherwise one line per server: the server id padded to a fixed width, its `enabled yes/no` state, and a status glyph (rendered as an emoji in the terminal - described here as connected, disabled, needs auth, or failed with the error text), followed by `tools: <n>` (the number of tools the server advertises, also printed for a server with none).
- A summary line: `<connected>/<total> server(s) connected`.
- A footer listing the subcommands: `/mcp discover`, `/mcp connect <id>`, `/mcp disconnect <id>`.

The display list is built from the same merged server set the connect path uses (`ragent_agent::plugin::plugin_mcp_server` so plugin-contributed servers appear here alongside the ones in `ragent.json`, and the live status is read from the session's MCP client.

`/mcp discover` produces no transcript output; it opens an interactive dialog (numbered selection list with a feedback line) fed by `McpClient::discover()`.

`/mcp connect <id>`:

- Missing id: `Usage: /mcp connect <id>`
- Id found: enables the server in the global ledger, connects it, registers its tools into the session registry, and reports the outcome.

`/mcp disconnect <id>`:

- Missing id: `Usage: /mcp disconnect <id>`

- Id found: disables the server in the global ledger, disconnects it, and reports the outcome. `McpToolWrapper::execute` then refuses to call that server's tools - even one still registered from an earlier connection - naming the command that re-enables it.

Related

- `mcp_tool` - the chat tool for calling MCP server tools
- `/reload mcp` - rescan MCP configuration after editing `ragent.json`
- `mcp` section in `ragent.json` - server definitions; `mcp.<id>.disabled` is the hard override that always wins over the ledger
- `/plugins list --mcp` - the live per-server tool inventory and registry names
- `/doctor` - includes MCP connectivity in its diagnostics